



Rúbrica de Evaluación - Semana 07

Repository Pattern



Competencias Evaluadas

Competencia	Peso
Implementación de Repository Pattern	35%
Repositorio Genérico (BaseRepository)	25%
Integración Service-Repository	25%
Testing con Repositories	15%



Criterios de Evaluación

1. Implementación de Repository Pattern (35%)

Excelente (100%)

- ✓ Repositorio abstrae completamente el acceso a datos
- ✓ Service no tiene imports de SQLAlchemy
- ✓ Métodos CRUD implementados correctamente
- ✓ Manejo de sesiones apropiado

Bueno (75%)

- ✓ Repositorio funcional con métodos básicos
- ⚠️ Alguna dependencia de SQLAlchemy en services
- ✓ CRUD funciona correctamente

Suficiente (50%)

- ✓ Repositorio implementado básicamente
- ⚠️ Separación de capas incompleta
- ⚠️ Algunos métodos faltantes

Insuficiente (< 50%)

- ✗ No hay separación clara
- ✗ Lógica de datos mezclada con negocio

2. Repositorio Genérico (25%)

Excelente (100%)

```
# Implementación esperada
class BaseRepository(Generic[T]):
    def __init__(self, db: Session, model: type[T]):
        self.db = db
        self.model = model

    def get_by_id(self, id: int) -> T | None: ...
    def get_all(self, skip: int, limit: int) -> list[T]: ...
    def create(self, obj: T) -> T: ...
```

```
def update(self, obj: T) -> T: ...
def delete(self, id: int) -> bool: ...
```

-  Uso correcto de Generics
-  Type hints completos
-  Métodos reutilizables
-  Repositorios específicos heredan correctamente

Bueno (75%)

-  BaseRepository funcional
-  Type hints incompletos
-  Herencia implementada

Suficiente (50%)

-  Clase base existe
-  Sin generics
-  Duplicación de código en repositorios

Insuficiente (< 50%)

-  Sin repositorio genérico
-  Código duplicado en cada repositorio

3. Integración Service-Repository (25%)

Excelente (100%)

```
# Service usando Repository
class TaskService:
    def __init__(self, task_repo: TaskRepository, user_repo: UserRepository):
        self.task_repo = task_repo
        self.user_repo = user_repo

    def create_task(self, data: TaskCreate) -> Task:
        # Validación de negocio
        user = self.user_repo.get_by_id(data.user_id)
        if not user:
            raise NotFoundError("User not found")

        # Delegación a repository
        task = Task(**data.model_dump())
        return self.task_repo.create(task)
```

-  Service recibe repositorios por inyección
-  Service NO conoce SQLAlchemy
-  Lógica de negocio en Service
-  Acceso a datos en Repository

Bueno (75%)

-  Integración funcional
-  Algo de lógica de datos en service

Suficiente (50%)

-  Service usa repository
-  Mezcla de responsabilidades

Insuficiente (< 50%)

- ❌ Service accede directamente a DB
- ❌ No usa repositories

4. Testing con Repositories (15%)

Excelente (100%)

```
# Test con repository mock
class FakeTaskRepository:
    def __init__(self):
        self.tasks = {}
        self._id = 1

    def create(self, task: Task) -> Task:
        task.id = self._id
        self.tasks[self._id] = task
        self._id += 1
        return task

def test_create_task():
    fake_repo = FakeTaskRepository()
    service = TaskService(task_repo=fake_repo)

    task = service.create_task(TaskCreate(title="Test"))

    assert task.id == 1
    assert task.title == "Test"
```

- ✅ Tests con fake repositories
- ✅ Service testado sin base de datos
- ✅ Cobertura de casos de éxito y error

Bueno (75%)

- ✅ Tests funcionales
- ⚠️ Pocos casos cubiertos

Suficiente (50%)

- ✅ Al menos 1 test con mock
- ⚠️ Tests dependen de BD real

Insuficiente (< 50%)

- ❌ Sin tests
- ❌ Tests no usan repositories



Escala de Calificación

Nivel	Rango	Descripción
🌟 Excelente	90-100%	Dominio completo del patrón
✅ Bueno	75-89%	Implementación sólida con mejoras menores
⚠️ Suficiente	60-74%	Cumple requisitos mínimos
❌ Insuficiente	< 60%	No cumple criterios básicos

Proyecto: Task Manager API

Requisitos Mínimos (60%)

- ☐ BaseRepository con métodos CRUD
- ☐ TaskRepository heredando de base
- ☐ UserRepository heredando de base
- ☐ TaskService usando repositorios
- ☐ Endpoints funcionando

Requisitos Completos (80%)

- ☐ Todos los anteriores
- ☐ Type hints completos
- ☐ Manejo de errores con excepciones custom
- ☐ Al menos 3 tests unitarios

Requisitos Avanzados (100%)

- ☐ Todos los anteriores
- ☐ Unit of Work implementado
- ☐ Tests con fake repositories
- ☐ Documentación en código

Checklist de Entrega

Estructura del Proyecto

```
task-manager/
├─ main.py
├─ config.py
├─ database.py
├─ models/
│  ├─ __init__.py
│  ├─ user.py
│  └─ task.py
├─ schemas/
│  ├─ __init__.py
│  ├─ user.py
│  └─ task.py
├─ repositories/           ← NUEVO
│  ├─ __init__.py
│  ├─ base.py             ← BaseRepository
│  ├─ user_repository.py
│  └─ task_repository.py
├─ services/
│  ├─ __init__.py
│  └─ task_service.py
├─ routers/
│  ├─ __init__.py
│  ├─ users.py
│  └─ tasks.py
└─ tests/                 ← NUEVO
```

```
├─ __init__.py
├─ test_task_service.py
```

Verificación

- ☐ Código ejecuta sin errores
- ☐ Tests pasan: `pytest tests/`
- ☐ Endpoints funcionan en `/docs`
- ☐ No hay imports de `sqlalchemy` en `services/`



Errores Comunes a Evitar

1. Repository con lógica de negocio

```
# ❌ MAL - validación en repository
class TaskRepository:
    def create(self, task: Task) -> Task:
        if task.due_date < date.today():
            raise ValueError("Invalid date") # ← NO aquí
        ...
```

2. Service con queries SQLAlchemy

```
# ❌ MAL - SQLAlchemy en service
class TaskService:
    def get_pending(self):
        return self.db.query(Task).filter(Task.done == False).all()
```

3. Repository sin tipado

```
# ❌ MAL - sin type hints
def get_by_id(self, id): # ← falta tipo de retorno
    return self.db.get(self.model, id)
```



Recursos de Apoyo

- [Repository Pattern - Fowler](#)
- [Python Generics](#)
- [pytest Documentation](#)